

实验三 Verilog基础

《电子技术实验2》

张翠翠

zhangcuicui@mail.xjtu.edu.cn

```
Module test(  
    input a,  
    input b,  
    input c,  
    input d,  
    input e,  
  
    output o1,  
    output o2,  
    output o3  
);  
  
assign o1 = a&b;  
assign o2 = c|d;  
assign o3 = ~e;  
  
endmoudle
```

◆ 一个设计实体中包含哪些要素？

- ✓ Module 模块定义
- ✓ 模块的端口定义
- ✓ 模块的内部逻辑实现

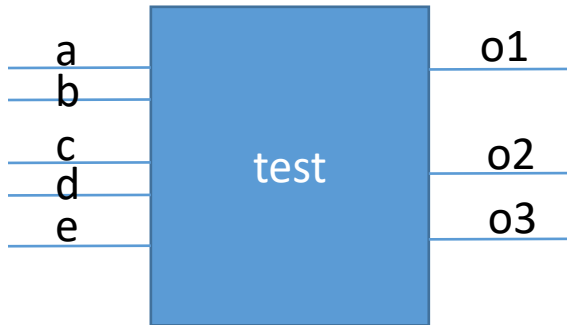
module(模块)

module是层次化设计的基本构件
逻辑描述放在**module**内部

```
Module test(  
    input a, b, c, d, e,  
    output o1, o2, o3  
);  
assign o1 = a&b;  
assign o2 = c|d;  
assign o3 = ~e;  
endmoudle
```

- **module**能够表示：
 - 物理块，如**IC**或**ASIC**单元
 - 逻辑块，如一个**CPU**设计的**ALU**部分
 - 整个系统
- 每一个模块的描述从关键词**module**开始，有一个名称（如**SN74LS74**，**DFF**，**ALU**等等），由关键词**endmodule**结束。

模块端口



端口在模块
后的括号中
列出

```
Module test(a,b,c,d,e,o1,o2,o3);
```

```
input a, b, c, d, e,  
output o1, o2, o3;
```

```
assign o1 = a&b;  
assign o2 = c|d;  
assign o3 = ~e;  
endmoudle
```

端口有三种类
型：input、
output、inout

- ✓ 端口定义了该模块的输入输出引脚pin
- ✓ 模块通过端口与外部通信

模块实例化

- 一个模块中可以包含其它模块，在一个模块中通过模块实例化来调用另一个模块。

```
Module test(  
    input a, b, c, d, e,  
    output o1, o2, o3  
);  
and u1(o1,a,b);  
or u2(o2,c,d);  
not u3(o3,e);  
endmodule
```

- ✓ 每个实例都有自己的名字(u1, u2, u3)。实例名是每个对象唯一的标记，通过这个标记可以查看每个实例的内部。
- ✓ 实例中端口的次序与模块定义的次序相同。
- ✓ 模块实例化与调用程序不同。每个实例都是模块的一个完全的拷贝，相互独立、并行。

整数常量和实数常量

- 整数的大小可以定义也可以不定义。

整数表示为: `<size>' <base><value>`

size : 表示占用的二进制位宽bit。缺省为32位

base: 数基, 可为2进制(b)、 8进制(o)、 10进制(d)、 16进制(h)
缺省为10进制

value: 是所选数基内任意有效数字, 包括X、 Z。

- 实数常量可以用十进制或科学表示法表示。

8'd12	8表示8bit, d表示是十进制。 该例表示值12
12'H83A	12表示12bit, H表示是十六进制。 该例表示值为16进制的83A
4'b0111	4表示4bit, b表示二进制。 该例表示二进制0111, 即7。

- 整数的大小可以定义也可以不定义。
 - 没有定义大小(size)的整数缺省为32bit
 - 缺省数基为十进制
 - 数基(base)和数字(16进制)中的字母无大小写之分
 - 当数值value大于指定的大小时，截去高位。
如 2' b1101表示的是2' b01
- 实数常量
 - 实数可用科学表示法或十进制表示
 - 科学表示法表示方式：
<尾数><e或E><指数>， 表示： 尾数 $\times 10^{\text{指数}}$
例：32E-4 0.0032; 4.1e3 4100.

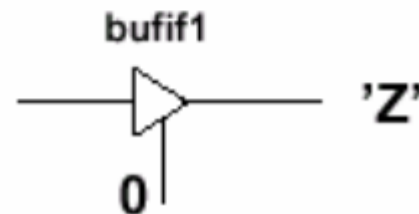
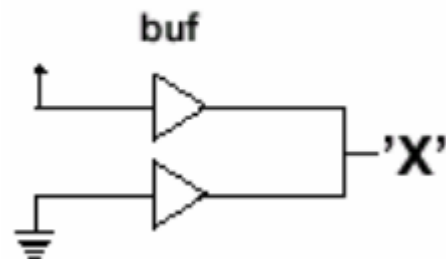
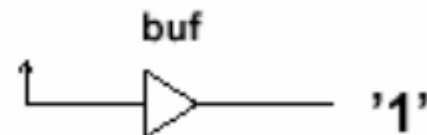
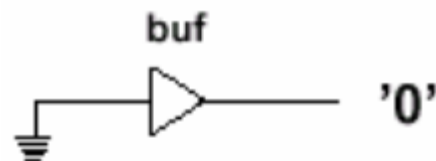
Verilog采用四值逻辑系统

0——逻辑假、低电平

1——逻辑真，高电平

‘X’ ——不确定的值

‘Z’ ——高阻态



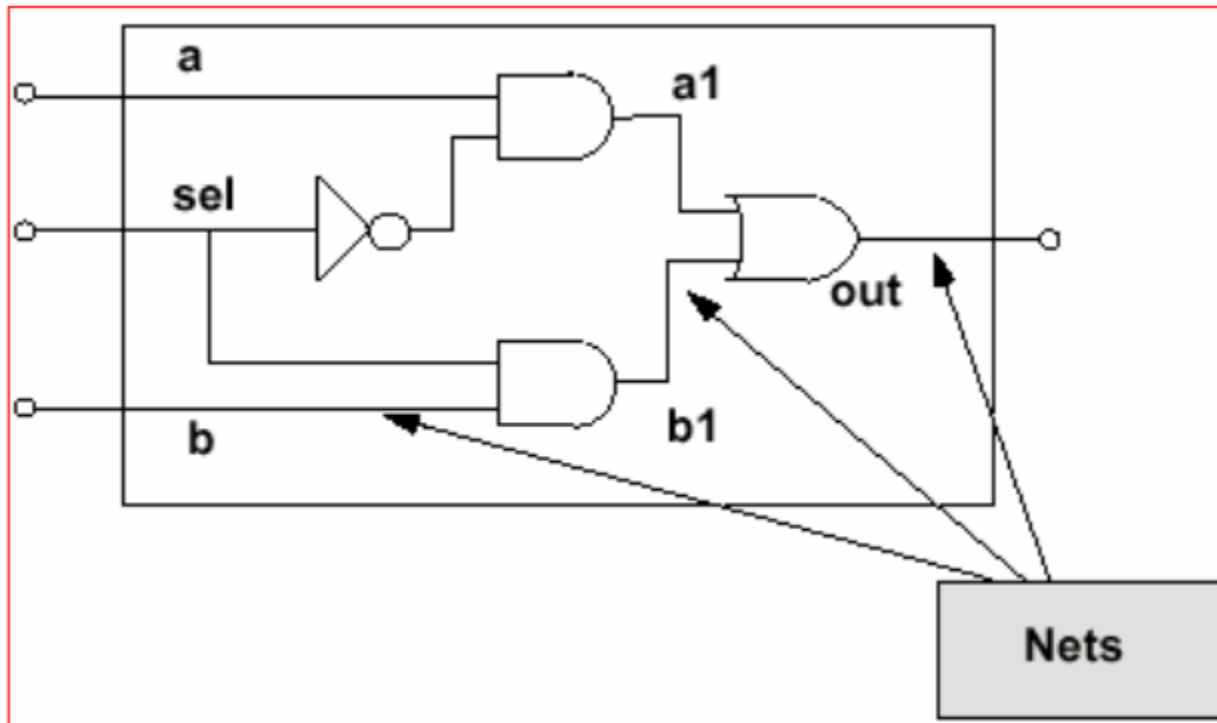
主要数据类型

■ Verilog主要有三类(class)数据类型:

- ✓ net(线网): 表示器件之间的物理连接
- ✓ register(寄存器): 表示抽象存储元件
- ✓ parameters(参数): 运行时的常数

net（线网）

- net 需要被持续的驱动，驱动它的可以是门和模块。
当net驱动器的值发生变化时， Verilog自动的将新值传送到net上。在例子中，线网out由or门驱动。当or门的输入信号置位时将传输到线网net上。



Net的类型



net类型	功 能
wire, tri	标准内部连接线(缺省)
supply1, supply0	电源和地
wor, trior	多驱动源线或
wand, triand	多驱动源线与
trireg	能保存电荷的net
tri1, tri0	无驱动时上拉/下拉

- 没有声明的net的缺省类型为 1 位(标量)wire类型。
- 但这个缺省类型可由下面的编译指导改变:
`default_nettype <nettype>

Net的类型

- **wire**类型是最常用的类型，只有连接功能。
- **wire**和**tri**类型有相同的功能。用户可根据需要将线网定义为**wire**或**tri**以提高可读性。

例如，可以用**tri**类型表示一个**net**有多个驱动源。或者将一个**net**声明为**tri**以指示这个**net**可以是高阻态Z(hign-impedance)。可推广至**wand**和**triand**、**wor**和**trior**

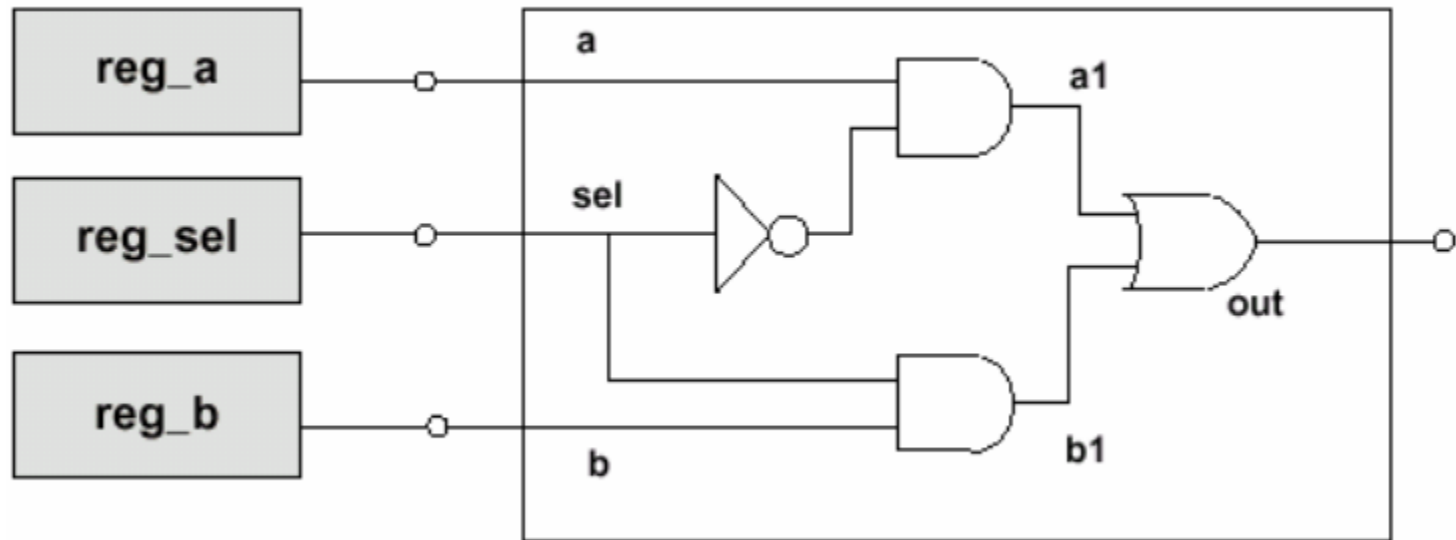
- **wand**、**wor**有线逻辑功能。
- **triereg**类型很象**wire**类型，但**triereg**类型在没有驱动时保持以前的值。这个值的强度随时间减弱。
- 修改**net**缺省类型的编译指导：

```
`default_nettype <nettype>
```

nettype不能是**supply1**和**supply0**。

寄存器类 (register)

- 寄存器类型在赋新值以前保持原值
- 寄存器类型大量应用于行为模型描述及激励描述。
在下面的例子中，**reg_a**、**reg_b**、**reg_sel**用于施加激励给2:1多路器。
- 用行为描述结构给寄存器类型赋值。给**reg**类型赋值是在过程块中。



寄存器类的类型

寄存器类型	功能
reg	可定义的非符号整数变量，可以是标量(1位)或矢量，是最常用的寄存器类型
integer	32位有符号整数变量，算术操作产生二进制补码形式的结果。通常用作不会由硬件实现的的数据处理。
real	双精度的带符号浮点变量，用法与integer相同。
time	64位非符号整数变量，用于仿真时间的保存与处理
realtime	与real内容一致，但可以作为实数仿真时间的保存与处理

Verilog中net和register声明语法

- net声明

`<net_type> [range] [delay] <net_name>[, net_name];`

net_type: net类型

range: 矢量范围，以[MSB: LSB]格式

delay: 定义与net相关的延时

net_name: net名称，一次可定义多个net，用逗号分开。

- 寄存器声明

`<reg_type> [range] <reg_name>[, reg_name];`

reg_type: 寄存器类型

range: 矢量范围，以[MSB: LSB]格式。只对reg类型有效

reg_name : 寄存器名称，一次可定义多个寄存器，用逗号分开

Verilog中net和register声明语法

- 举例:

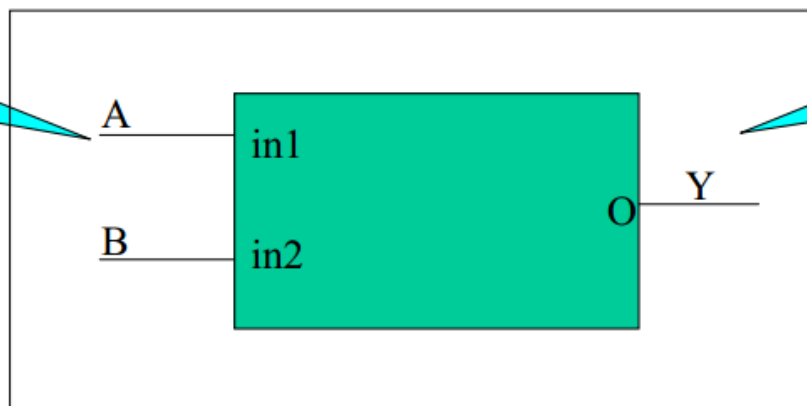
```
reg a; //一个标量寄存器
wand w; // 一个标量wand类型net
reg [3: 0] v; // 从MSB到LSB的4位寄存器向量
reg [7: 0] m, n; // 两个8位寄存器
tri [15: 0] busa; // 16位三态总线
wire [0: 31] w1, w2; // 两个32位wire, MSB为bit0
```

选择正确的数据类型

输入端口可以由
net/register驱动，但输入
端口只能是net

双向端口输入/输出只
能是net类型

输出端口可以是
net/register类型，输出
端口只能驱动net



```
module top;
wire y;
reg a, b;
    DUT u1 (y, a, b);
    initial begin
        a = 0; b = 0;
        #5 a = 1;
    end
endmodule
```

在过程块中只能给
register类型赋值

```
module DUT (Y, A, B);
output Y;
input A, B;
wire Y, A, B;
    and (Y, A, B);
endmodule
```

若Y, A, B说明为
reg则会产生错误。

选择数据类型时常犯的错误

- 信号可以分为端口信号和内部信号。出现在端口列表中的信号是端口信号，其它的信号为内部信号。
- 对于端口信号，输入端口只能是**net**类型。输出端口可以是**net**类型，也可以是**register**类型。若输出端口在过程块中赋值则为**register**类型；若在过程块外赋值(包括实例化语句)，则为**net**类型。
- 内部信号类型与输出端口相同，可以是**net**或**register**类型。判断方法也与输出端口相同。若在过程块中赋值，则为**register**类型；若在过程块外赋值，则为**net**类型。

下面所列是常出的错误及相应的错误信息(error message)

- 用过程语句给一个**net**类型的或忘记声明类型的信号赋值。
信息: `illegal ..... assignment.`
- 将实例的输出连接到声明为**register**类型的信号上。
信息: `<name> has illegal output port specification.`
- 将模块的输入信号声明为**register**类型。
信息: `incompatible declaration, <signal name> .....`

参数（parameters）

- 用参数声明一个可变常量，常用于定义延时及宽度变量。
- 参数定义的语法：parameter
<list_of_assignment>;
 - 可一次定义多个参数，用逗号隔开。
 - 参数的定义是局部的，只在当前模块中有效。
 - 参数定义可使用以前定义的整数和实数参数。

```
module mod1( out, in1, in2);
```

```
  . . .
```

```
  Parameter IDLE = 4 'b0000;
```

```
endmodule
```

Verilog操作符

操作符类型	符号	
连接及复制操作符	{ } { }	最高 ↑ 优先级 ↓ 最低
一元操作符	! ~ & ^	
算术操作符	* / % + -	
逻辑移位操作符	<< >>	
关系操作符	> < >= <=	
相等操作符	== === != !==	
按位操作符	& ^ ~^ 	
逻辑操作符	&& 	
条件操作符	? :	

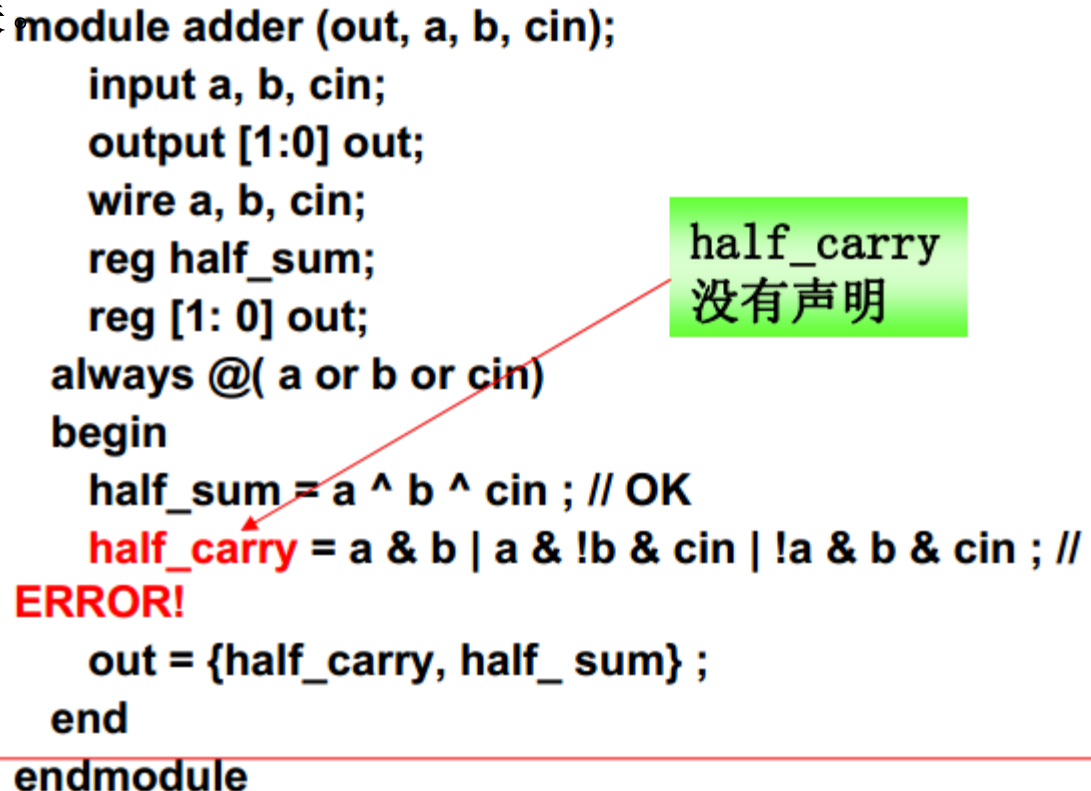
过程 (procedural) 块

- 过程块是行为模型的基础。
- 过程块有两种：
 - `initial`块，只能执行一次。不能综合，用在仿真中
 - `always`块，循环执行
- 过程块中有下列部件
 - 过程赋值语句：在描述过程块中的数据流
 - 高级结构（循环，条件语句）：描述块的功能
 - 时序控制：控制块的执行及块中的语句。

过程赋值(procedural assignment)

- 在过程块中的赋值称为过程赋值。
- 在过程赋值语句中表达式左边的信号必须是寄存器类型（如**reg**类型）
- 在过程赋值语句等式右边可以是任何有效的表达式，数据类型也没有限制。
- 如果一个信号没有声明则缺省为**wire**类型。使用过程赋值语句给**wire**赋值会产生错误

```
module adder (out, a, b, cin);  
    input a, b, cin;  
    output [1:0] out;  
    wire a, b, cin;  
    reg half_sum;  
    reg [1: 0] out;  
    always @( a or b or cin)  
    begin  
        half_sum = a ^ b ^ cin ; // OK  
        half_carry = a & b | a & !b & cin | !a & b & cin ; //  
        ERROR!  
        out = {half_carry, half_sum} ;  
    end  
endmodule
```



边沿敏感时序

- 时序控制@可以用在RTL级或行为级组合逻辑或时序逻辑描述中。
- 可以用关键字`posedge`和`negedge`限定信号敏感边沿。敏感表中可以有多个信号，用关键字`or`连接。

```
module reg_adder (out, a, b, clk);  
    input clk;  
    input [2: 0] a, b;  
    output [3: 0] out;  
    reg [3: 0] out;  
    reg [3: 0] sum;  
    always @( a or b) // 若a或b发生任何变化，执行  
        #5 sum = a + b;  
    always @( negedge clk) // 在clk下降沿执行  
        out = sum;  
endmodule
```

边沿敏感时序

例：分频器。

对输入时钟做分频，以得到更低频率的时钟。

输入信号：clk（高频时钟），输出信号clk_2（低频时钟）。

```
reg [3:0] count;
always @(posedge clk) begin
    count <= count + 4'h1;
end

reg clk_2;
always @(posedge clk) begin
    if(count == 4'h1) begin
        clk_2 <= ~clk_2;
    end
    else begin
        clk_2 <= clk_2;
    end
end
```

条件语句（if分支语句）

例1：二选一数控开关。

输入信号a,b,sel, 输出信号y

当sel为1时, 选择a路信号输出给y;

当sel为0时, 选择b路信号输出给y。

```
always @(a or b or sel) begin
    if(sel == 1'b1) y <= a;
    else           y <= b;
end
```

例2：带使能的二选一数控开关。

输入信号a,b,sel,en, 输出信号y

当en为1时, 输出y符合例1中的规则;

当en为0时, y恒输出0。

```
always @(*) begin
    if(en == 1'b1) begin
        if(sel == 1'b1) y <= a;
        else           y <= b;
    end
    else begin
        y <= 0;
    end
end
```

描述方式:

if (表达式)

begin

.....

end

else

begin

.....

end

- 可以多层嵌套。在嵌套if序列中, else和前面最近的if相关。
- 为提高可读性及确保正确关联, 使用begin...end块语句指定其作用域。

条件语句-case语句

- 例3：数据分配器

输入信号a,b,i, 输出信号y0,,y1,y2,y3;

当ab为00时, i输出给y0, y1y2y3输出0;

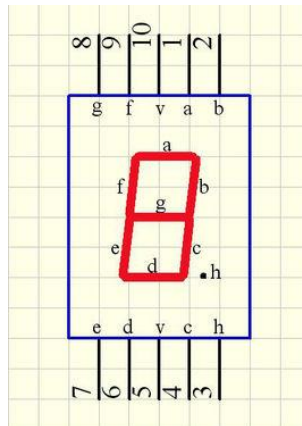
当ab为01时, i输出给y1, y0y2y3输出0;

当ab为10时, i输出给y2, y0y1y3输出0;

当ab为11时, i输出给y3, y0y1y2输出0。

```
always @(*) begin
    case({a,b})
        2'b00: {y3,y2,y1,y0} <= {3'b000,i};
        2'b01: {y3,y2,y1,y0} <= {2'b00,i,1'b0};
        2'b10: {y3,y2,y1,y0} <= {1'b0,i,2'b00};
        2'b11: {y3,y2,y1,y0} <= {i,3'b000};
        default: ;
    endcase
end
```

- 例4：7段译码管



```
always @(k) begin
    case (k)
        4'b0000: seg7out = 7'b0000001;
        4'b0001: seg7out = 7'b1001111;
        4'b0010: seg7out = 7'b0010010;
        4'b0011: seg7out = 7'b0000110;
        4'b0100: seg7out = 7'b1001100;
        4'b0101: seg7out = 7'b0100100;
        4'b0110: seg7out = 7'b0100000;
        4'b0111: seg7out = 7'b0001111;
        4'b1000: seg7out = 7'b0000000;
        4'b1001: seg7out = 7'b0000100;
        4'b1010: seg7out = 7'b0001000;
        4'b1011: seg7out = 7'b1100000;
        4'b1100: seg7out = 7'b0110001;
        4'b1101: seg7out = 7'b1000010;
        4'b1110: seg7out = 7'b0110000;
        4'b1111: seg7out = 7'b0111000;
    endcase
end
```

循环(looping)语句

■ 有四种循环语句：

repeat: 将一块语句循环执行确定次数。

repeat (次数表达式) <语句>

while: 在条件表达式为真时一直循环执行

while (条件表达式) <语句>

forever: 重复执行直到仿真结束

forever <语句>

for: 在执行过程中对变量进行计算和判断，在条件满足时执行

for(赋初值; 条件表达式; 计算) <语句>

➤ 不常用，只有*for*语句是可以被综合的，其它三种只在仿真中使用。

持续赋值 (continuous assignment)

- 可以用持续赋值语句描述组合逻辑，代替用门及其连接描述方式。
- 持续赋值在过程块外部使用。
- 持续赋值用于net驱动。
- 持续赋值只能在等式左边有一个简单延时说明。
 - 只限于在表达式左边用#delay形式
- 持续赋值可以是显式或隐含的。

语法：

`<assign> [#delay] [strength] <net_name> = <expressions>;`

```
wire out;
```

```
assign out = a & b; // 显式
```

```
wire inv = ~in; // 隐含
```

实验任务一：

- 完成例2：带使能的二选一数控开关 代码并仿真
- 完成例3：数据分配器 代码并仿真

实验任务二：计数器

- 输入信号
 - 计数脉冲信号clk，清零信号clr；
 - 输出信号
 - 步数y。
- 1.考虑步数y的位宽，根据以上描述先写出module模块的定义（包含端口列表和端口信号定义）
 - 2.考虑步数y的类型（wire or reg）
 - 3.考虑用过程赋值（always）还是持续赋值（assign）
 - 4.如果用过程赋值，考虑敏感列表里应包含哪些信号？
 - 5.写出完整的代码并仿真。